ARTIFICIAL INTELLIGENCE

Lecture Set 14
Regression

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
6. Local Search
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Knowledge Representation and Logical Agents
10. Uncertainty, Conditional Probability, Bayes Rule
11. Bayesian Networks, Naïve Bayes Classifier
12. Hidden Markov model, Markov Decision Process
13. **Machine Learning (Supervised, Unsupervised, Reinforcement Learning)**

Machine Learning definition

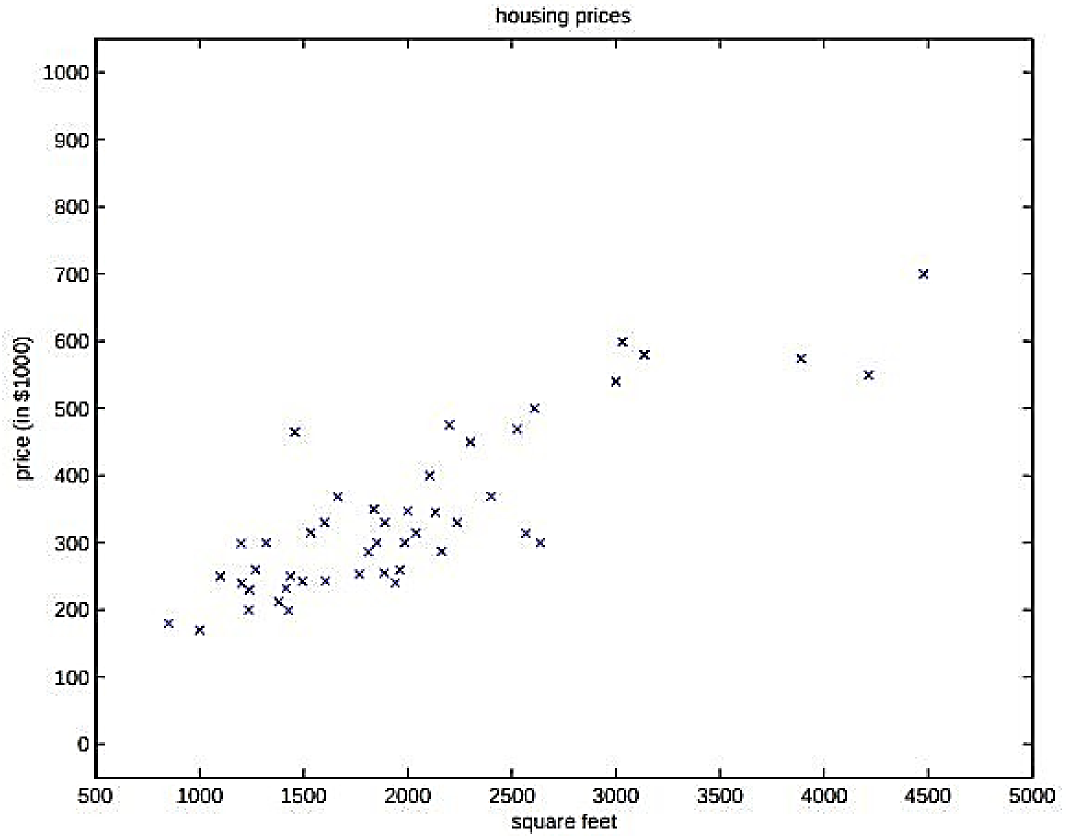
- Arthur Samuel (1959). Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed.
- Tom Mitchell (1998) Well-posed Learning Problem: A computer program is said to *learn* from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

Supervised Learning

Suppose we are given a **dataset** for **supervised learning**:

Living area (feet^2)	Price (1000\$s)
2104	400
1600	330
2400	369
1416	232
3000	540
$\vdots$	$\vdots$

We can plot this data …



Decision making …

Given data like this, how can we learn to **predict** the **prices** of other houses, as a **function** of the size of their **living areas**?

Notions for variables

Input variable(s) : $\mathbf{x^{(i)}}$ **---** (living area in the previous example) --- also called input features

Output variable: $\mathbf{y^{(i)}}$ --- target variable we are trying to predict --- (price)

A pair $\mathbf{(x^{(i)}, y^{(i)})}$ is called a training example

$\mathbf{\{(x^{(i)},y^{(i)});i = 1, \ldots, m\}}$ -- a list of m training examples, training set

- **X** : space of input variables
- **Y** : space of output variables
- **X, Y** $\in$ **R** - Real number - one dimensional

Supervised learning problem

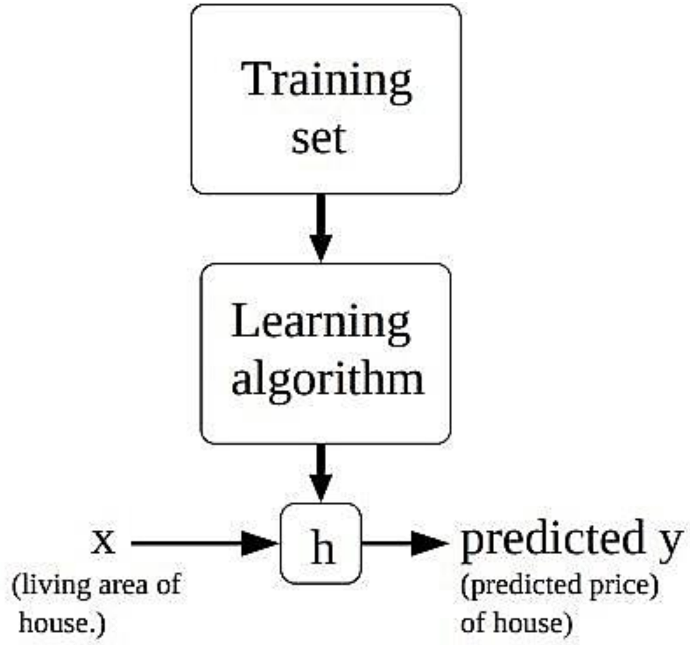
Our goal is, given a training set, to learn a function **h: X → Y**

Such that **h** is a “good predictor” of the corresponding value of y

This function **h** is called hypothesis

Different functions **h** constitute **hypotheses space of models**

Learning function h, the hypothesis



Continuous target variables

When the target variable that we are going to predict is **continuous**

As in our example of predicting **house price**

We call the (supervised) learning problem a **regression** problem

In contrast to **classification** problems, where the target variable can take on only a small number of **discrete** values.

Linear Regression

Example : Price is function of two variables or features: Living area (feet^2) and #bedrooms

x are the two-dimensional vectors in $\mathbf{R}^2$. $\mathbf{x_1^{(i)}, x_2^{(i)}}$

Living area (feet^2)	#bedrooms	Price (1000$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮

Variable Selection, Feature Selection

When designing a learning problem, it is up to you to select variables/ features that you deem important.

The selected, devised features should aid in predicting the target variable

Representing hypothesis function

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

θ_i's are the parameters (also called weights)

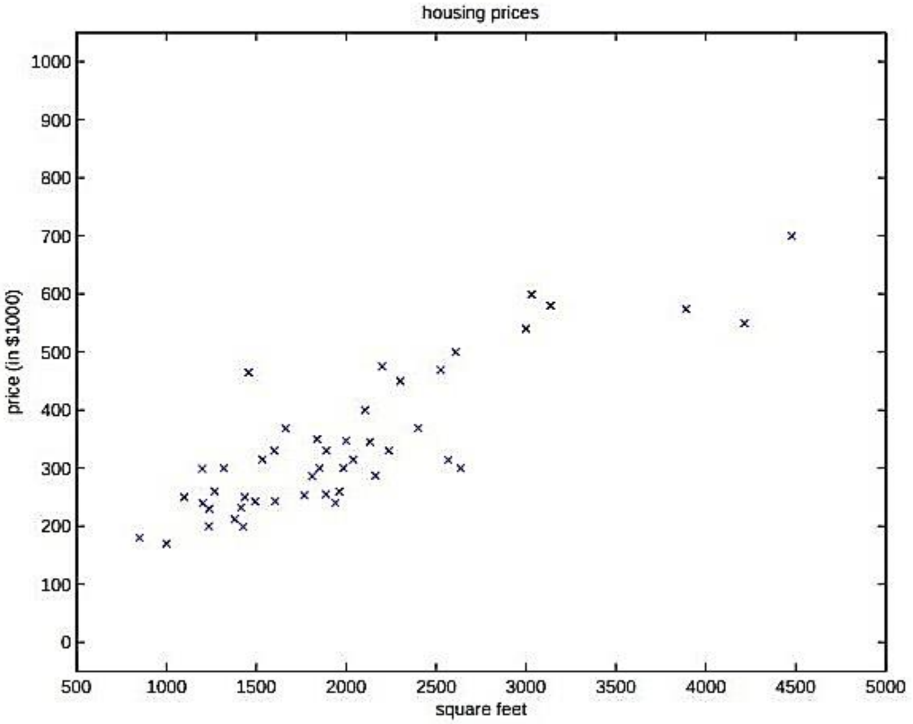
θ_i's parameterize the space of linear functions mapping from X to Y

Simplifying the notation: letting x_0 to 1 (this is the intercept term):

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x,$$

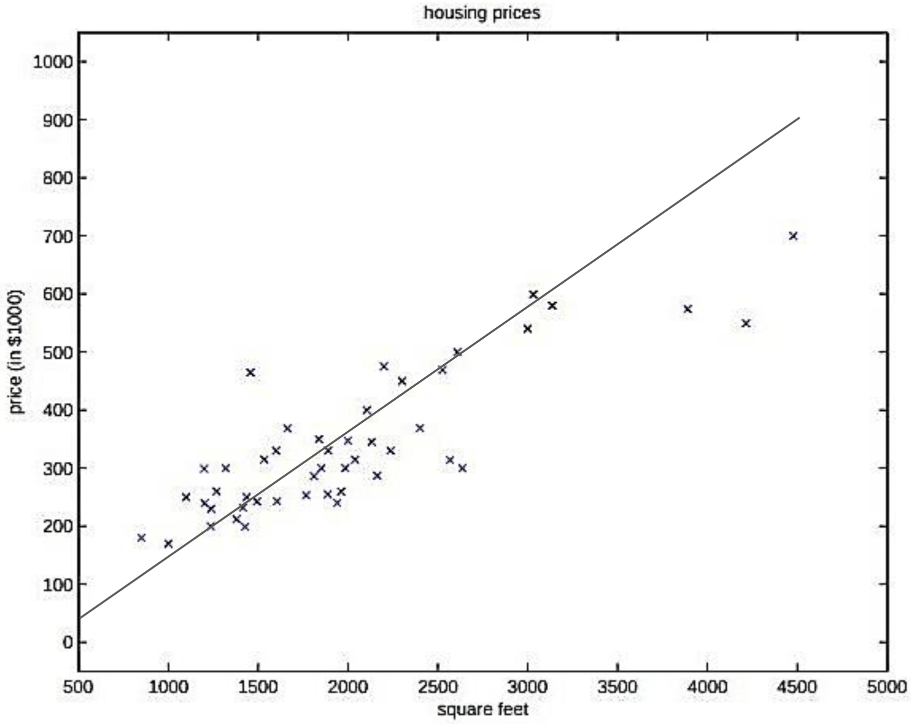
Learning the parameters

Now, given a training set, how do we pick, or learn, the parameters θ?



Learning the parameters

h(x) predictions should be close to the training set/ data



Learning the parameters

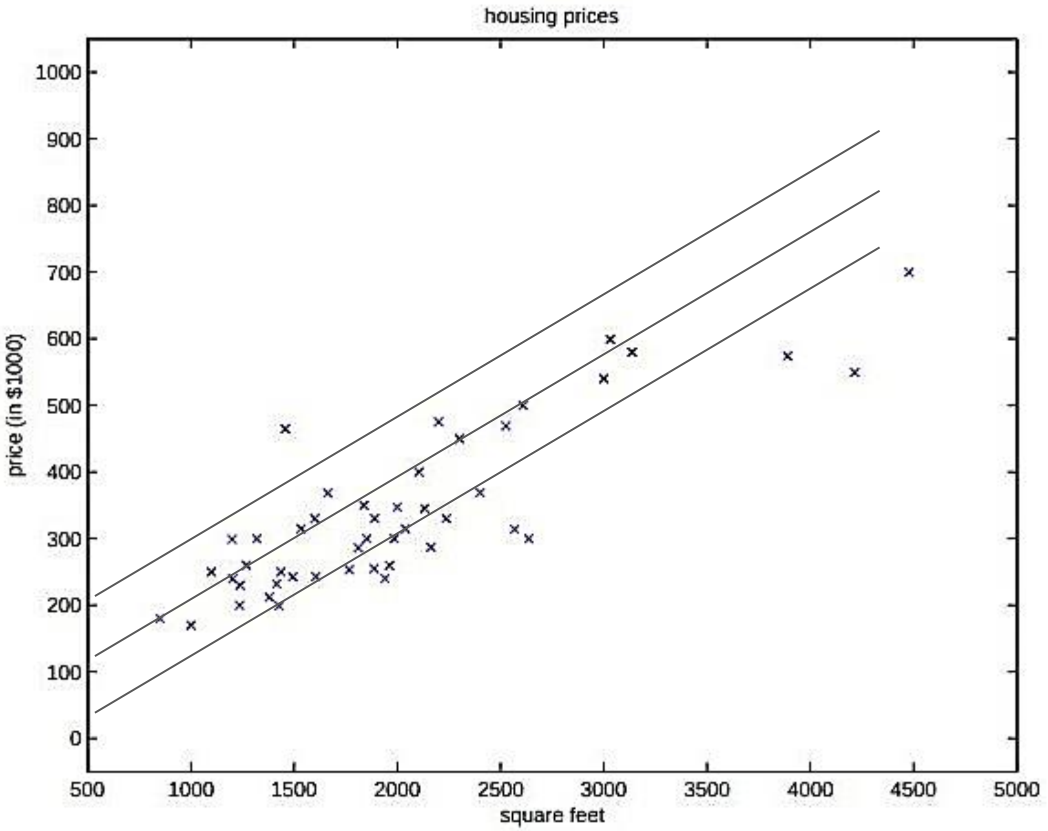
To measure how close is h(x) to y - the target variable,

We define a **cost function**, which quantify for each of θ's, how close to the h(x^i)'s are to the corresponding y^i's:

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2.$$

The above cost function is called **least-squares cost function**

Ordinary least squares regression model



LMS algorithm (least-mean squares)

We want to choose **θ** so as to minimize **J(θ)**

We will search for the best set values for **θ**

We start with an **initial guess of θ**, and

Repeatedly **change θ to make J(θ) smaller**

Gradient Descent Algorithm

Gradient Descent algorithm start with an initial θ, and repeatedly performs the update:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$

(This update is simultaneously performed for all values of j = 0, . . . , n.)

Here, **α** is called the learning rate

J(θ) is the partial derivative

Partial derivative of cost function (for one training example)

$$\begin{aligned}
\frac{\partial}{\partial\theta_j} J(\theta) &= \frac{\partial}{\partial\theta_j} \frac{1}{2} \left(h_\theta(x) - y\right)^2 \\
&= 2 \cdot \frac{1}{2} \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial\theta_j} \left(h_\theta(x) - y\right) \\
&= \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial\theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y\right) \\
&= \left(h_\theta(x) - y\right) x_j
\end{aligned}$$

Update rule: (for single training example)

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}.$$

The rule is called the LMS update rule (LMS stands for “least mean squares”)

The magnitude of the update is proportional to the error term

Update rule: (for complete training set)

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)} \qquad \text{(for every } j\text{)}.$$

}

Gradient descent on original cost function J.

Computes gradient of after scanning the complete training set, called batch gradient descent

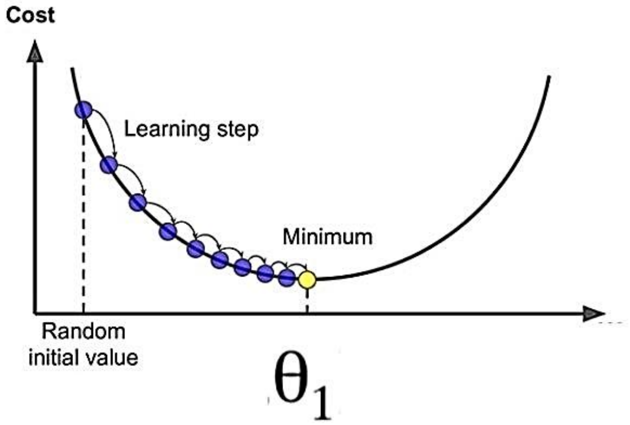
Update rule: univariate case - single parameter

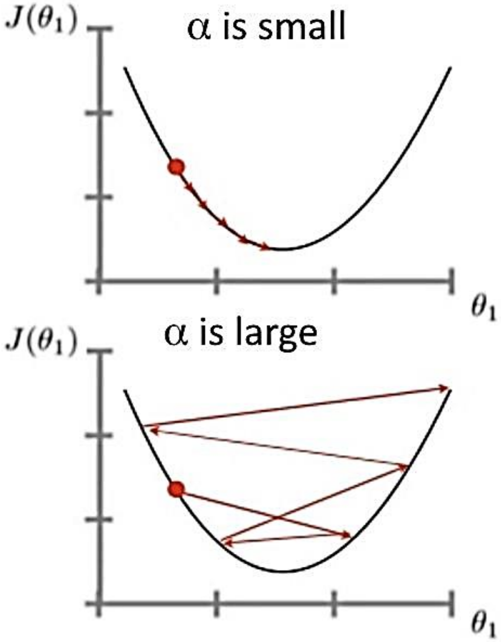
repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

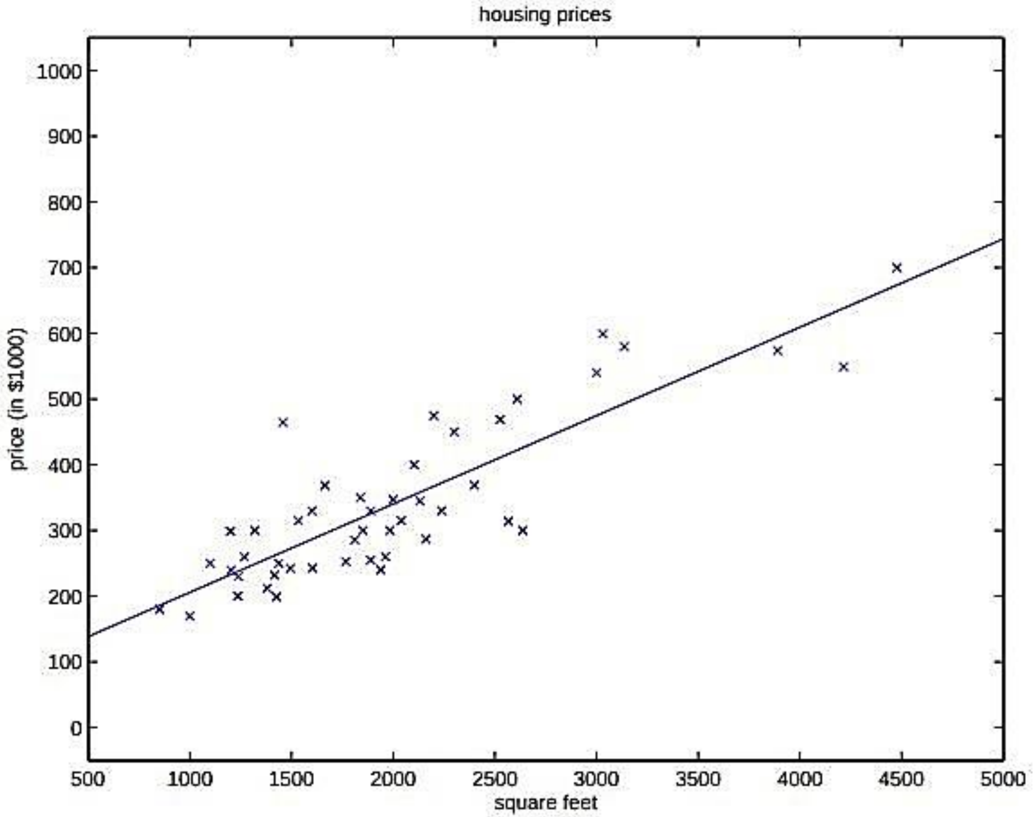
$$(\text{for } j = 1 \text{ and } j = 0)$$

}

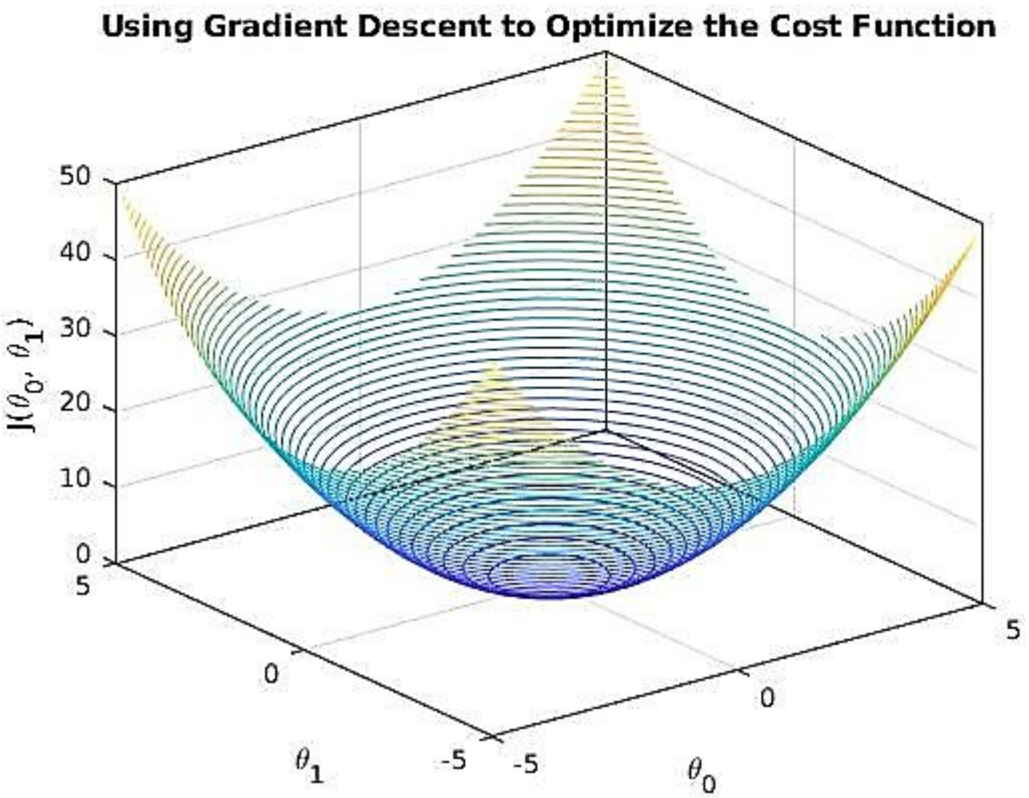




Result of LMS algorithm



Update rule: two parameters



Stochastic gradient descent

Whereas batch gradient descent has to scan through the entire training set before taking a single step—a costly operation if m is large

Stochastic gradient descent can start making progress right away, and continues to make progress with each example it looks at.

Often, stochastic gradient descent gets θ "close" to the minimum much faster than batch gradient descent.

Stochastic gradient descent

Loop {

for i=1 to m, {

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)} \qquad \text{(for every } j\text{).}$$

}

}

Linear Regression

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

Homework

Formula for calculating slope and intercept

$$\hat{\beta} = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2}$$

$$\hat{\alpha} = \overline{y} - \hat{\beta}\,\overline{x}\ ,$$

- Predict the price of pizza when its diameter is 20 inches.

Diameter(X) In Inches	Price(Y) In Dollars
8	10
10	13
12	16

X	Y
1	1
2	2
3	1.3
4	3.75
5	2.25

- Let us consider an example where the five weeks' sales data (in Thousands) is given as shown in Table.
- Apply linear regression technique to predict the 7th and 12th week sales.

x_i (Week)	y_j (Sales in Thousands)
1	1.2
2	1.8
3	2.6
4	3.2
5	3.8

x1 Product 1 Sales	x2 Product 2 Sales	Y Weekly Sales
1	4	1
2	5	6
3	8	8
4	2	12

$Y = 2.746X_1 - 1.52X_2 + 0.212$

$Y = 0.425X + 0.785$

X	Y	XY	X^2
1	1	1	1
2	2	4	4
3	1.3	3.9	9
4	3.75	15	16
5	2.25	11.25	25
15	10.3	35.15	55

X_1	X_2	Y	X_1^2	X_2^2	X_1X_2	X_1Y	X_2Y
3	8	-3.7	9	64	24	-11.1	-29.6
4	5	3.5	16	25	20	14	17.5
5	7	2.5	25	49	35	12.5	17.5
6	3	11.5	36	9	18	69	34.5
2	1	5.7	4	1	2	11.4	5.7
20	24	19.5	90	148	99	95.8	45.6

$$m = \frac{n\sum XY - \sum X \sum Y}{n(\sum X^2) - (\sum X)^2}$$

$$m = \frac{5(35.15) - (15)(10.3)}{5(55) - (15)^2}$$

$$m = \frac{21.25}{50} = 0.425$$

$$c = \bar{Y} - m\bar{X}$$

$$c = \frac{10.3}{5} - (0.425)\left(\frac{15}{5}\right) = 0.785$$

$$b_1 = \frac{(\sum X_2^2)(\sum X_1Y) - (\sum X_1X_2)(\sum X_2Y)}{(\sum X_1^2)(\sum X_2^2) - (\sum X_1X_2)^2} = \frac{(148)(95.8) - (99)(45.6)}{(90)(148) - (99)^2}$$

$$b_1 = 2.746$$

$$b_2 = \frac{(\sum X_1^2)(\sum X_2Y) - (\sum X_1X_2)(\sum X_1Y)}{(\sum X_1^2)(\sum X_2^2) - (\sum X_1X_2)^2} = \frac{(90)(45.6) - (99)(95.8)}{(90)(148) - (99)^2}$$

$$b_2 = -1.52$$

$$a = \bar{Y} - b_1\bar{X}_1 - b_2\bar{X}_2 = \frac{19.5}{5} - (2.746)\left(\frac{20}{5}\right) + (1.52)\left(\frac{24}{5}\right) = 0.212$$

Linear Regression 2 independent variable

SUBJECT	Y	X_1	X_2	X_1X_1	X_2X_2	X_1X_2	X_1Y	X_2Y
1	-3.7	3	8	9	64	24	-11.1	-29.6
2	3.5	4	5	16	25	20	14	17.5
3	2.5	5	7	25	49	35	12.5	17.5
4	11.5	6	3	36	9	18	69	34.5
5	5.7	2	1	4	1	2	11.4	5.7
Σ	19.5	20	24	90	148	99	95.8	45.6

$$a = \overline{Y} - b_1\overline{X}_1 - b_2\overline{X}_2$$

$$b_1 = \frac{(\sum x_2^2)(\sum x_1y) - (\sum x_1x_2)(\sum x_2y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1x_2)^2}$$

$$b_2 = \frac{(\sum x_1^2)(\sum x_2y) - (\sum x_1x_2)(\sum x_1y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1x_2)^2}$$

$$b_1 = \frac{(\sum x_2^2)(\sum x_1y) - (\sum x_1x_2)(\sum x_2y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1x_2)^2} = \frac{32.8 * 17.8 - 3 * (-48)}{10 * 32.8 - 3 * 3} = 2.28$$

$$b_2 = \frac{(\sum x_1^2)(\sum x_2y) - (\sum x_1x_2)(\sum x_1y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1x_2)^2} = \frac{10 * (-48) - 3 * 17.8}{10 * 32.8 - 3 * 3} = -1.67$$

$$a = \overline{Y} - b_1\overline{X}_1 - b_2\overline{X}_2 = \frac{19.5}{5} - \frac{2.28 * 20}{5} - \frac{-1.67 * 24}{5} = 2.796$$

Final Regression equation or Model is:

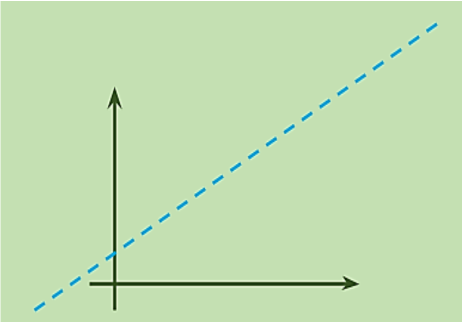
$$\boldsymbol{Y = 2.796 + 2.28\, x_1 - 1.67 x_2}$$

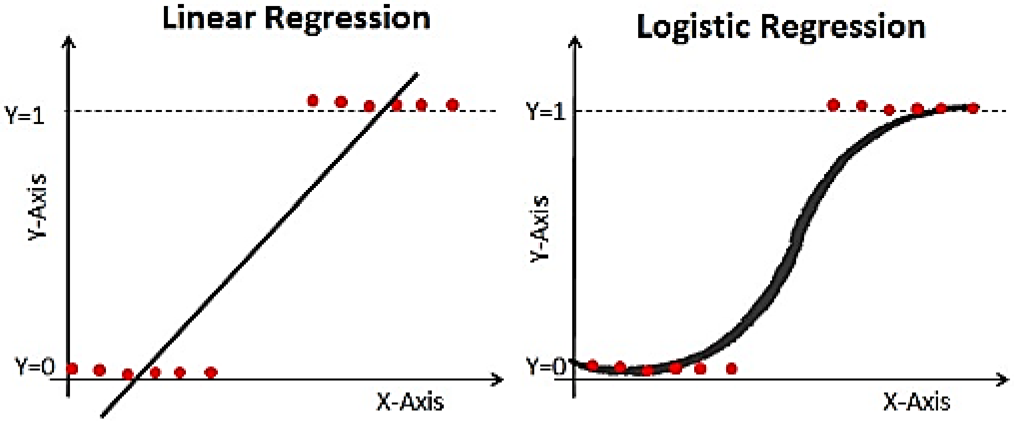
$$Now\ given \quad x_1 = 3 \quad and \quad x_2 = 2 \qquad Y = ?$$

$$Y = 2.796 + 2.28 * 3 - 1.67 * 2$$
$$= \mathbf{6.296}$$

Problem in Linear Regression

- Not all data is linear
- As x increases y tends to approach infinity.
- Not the case in reality.





$$h_\theta(x) = \theta_0 + \theta_1 x_1$$

$$p = \frac{1}{1+e^{-z}}$$

Sigmoid function brings non-linearity. Based on a threshold value, classes can be classified using the probability value we get after applying sigmoid function.

Logistic Regression – Solved Example 1

- The dataset of pass or fail in an exam of 5 students is given in the table.
- Use logistic regression as classifier to answer the following questions.

1. Calculate the probability of pass for the student who studied 33 hours.

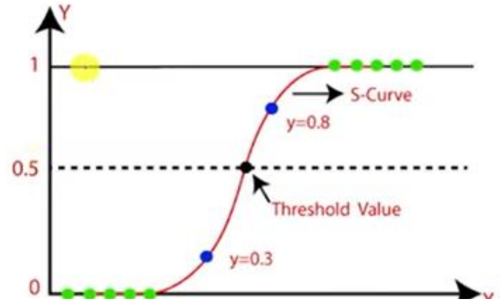
Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

- We use Sigmoid Function in logistic regression

Assume the model suggested by the optimizer for odds of passing the course is,

$$\log(odds) = -64 + 2 * hours$$

- $s(x) = \frac{1}{1+e^{-x}}$



Logistic Regression – Solved Example 1

1. Calculate the probability of pass for the student who studied 33 hours.

$$s(x) = \frac{1}{1 + e^{-x}}$$

- $p = \frac{1}{1+e^{-z}}$
- $z = -64 + 2 * 33 = -64 + 66 = 2$
- $p = \frac{1}{1+e^{-2}} = 0.88$
- That is, if student studies 33 hours, then there is **88% chance** that the student will pass the exam

Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

$$\log(odds) = z = -64 + 2 * hours$$

https://www.youtube.com/watch?v=2C8IqOLO1os